

# Adders

*MDE - Advanced Engineering Topics, ITESO*

*Gomez Enriquez, Gerardo*

[gerardo.gomez@iteso.mx](mailto:gerardo.gomez@iteso.mx)

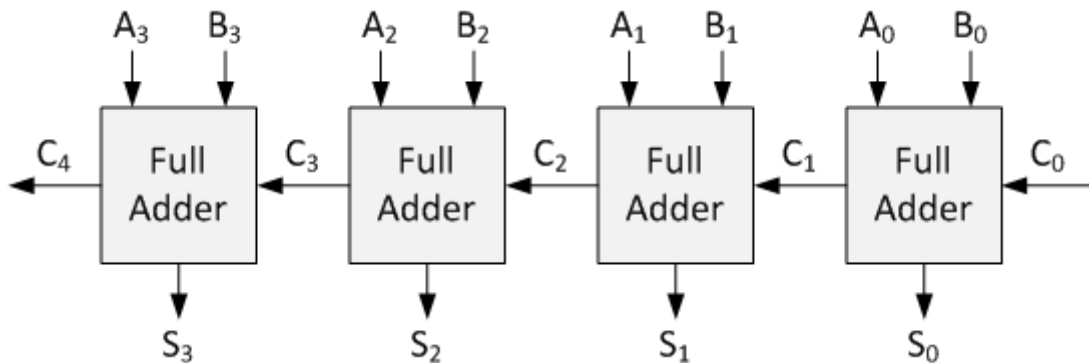
*Guadalajara, Mexico*

## Architecture and considerations

### Ripple Carry Adder (RCA)

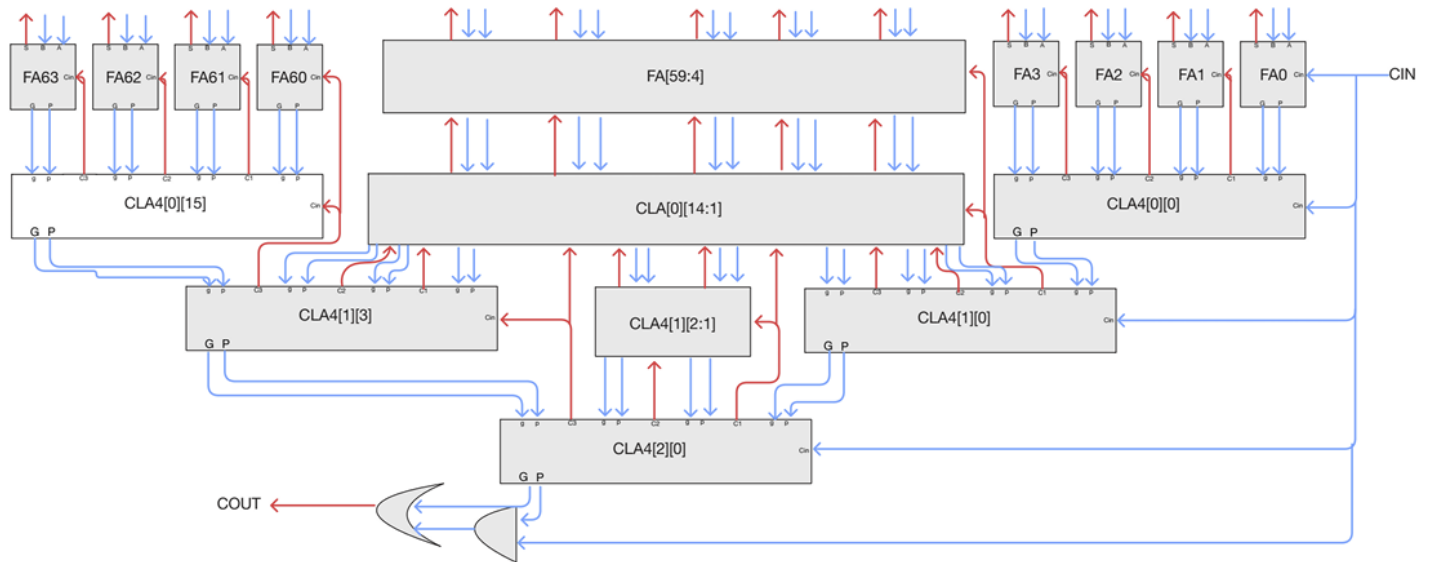
The RCA is a structurally simple architecture whose critical path scales linearly with width — every additional bit adds one full-adder stage to the carry chain because it propagates through all full-adders. Considerations:

- Advantages: Simple, minimal area
- Disadvantages: Lineal delay. Not suitable for modern CPUs with wide additions.



## Carry Look-ahead Adder (CLA)

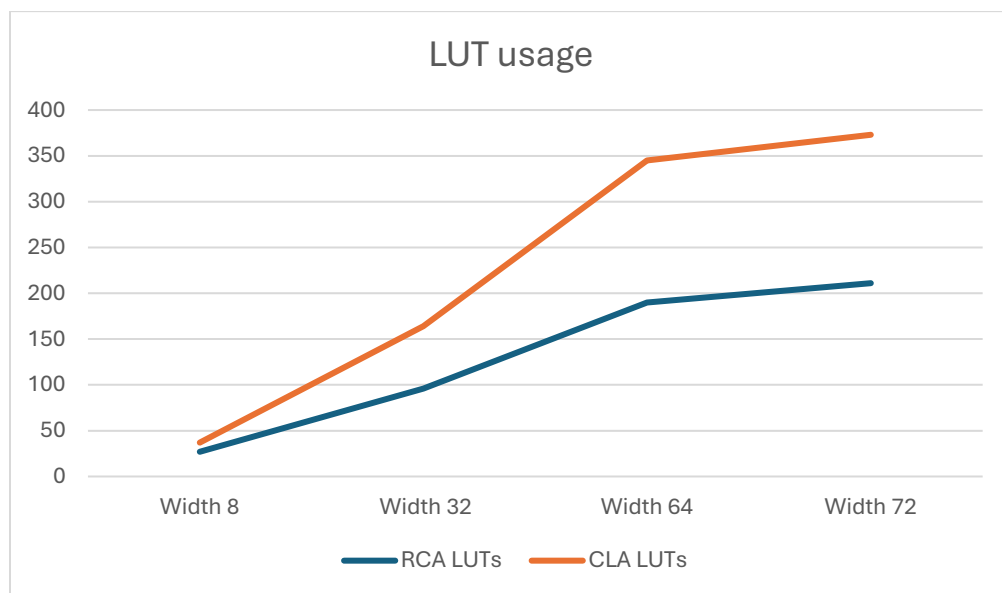
The CLA trades area for speed by computing carry signals in parallel. The logic depth of the carry out computation grows logarithmically since the Look-Ahead Carry Units (LCUs) are connected in a hierarchical way (tree of carries). By using 4-bit wide LCUs, the delay grows by  $\log_4(N)$ .



## Area / resource usage analysis

### LUT usage

| Width | RCA LUTs | CLA LUTs | CLA overhead vs RCA |
|-------|----------|----------|---------------------|
| 8     | 27       | 37       | +37%                |
| 32    | 96       | 164      | +71%                |
| 64    | 190      | 345      | +82%                |
| 72    | 211      | 373      | +77%                |

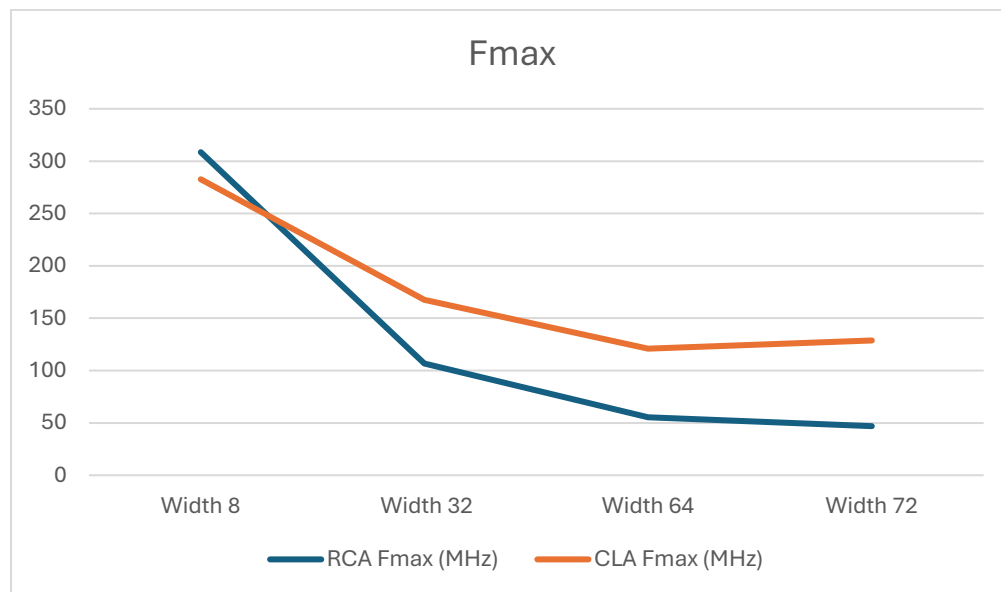


- The RCA consistently uses fewer LUTs than the CLA across all widths, which is expected — the CLA requires extra logic to compute the carry generate/propagate signals in parallel.
- The CLA overhead grows from ~37% at width 8 up to ~82% at width 64, then slightly drops to ~77% at width 72. This reflects the increased carry lookahead logic that is added at wider widths.
- Both adders scale roughly linearly with width, confirming the CLA trades area for speed.

## Timing / latency analysis

### Fmax

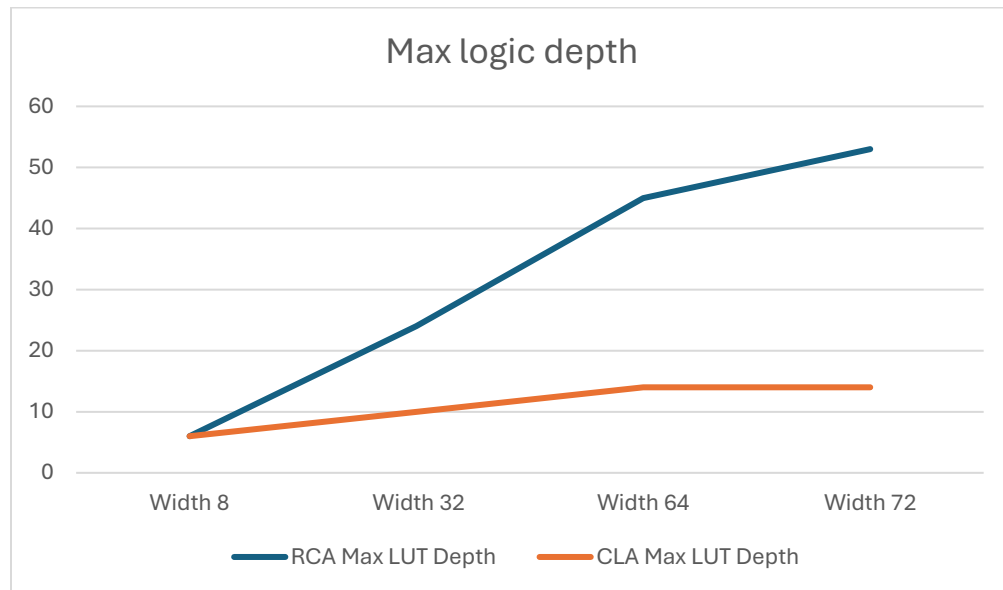
| Width | RCA Fmax (MHz) | CLA Fmax (MHz) | CLA speedup vs RCA |
|-------|----------------|----------------|--------------------|
| 8     | 308.55         | 282.65         | -8.4%              |
| 32    | 106.78         | 167.42         | +56.8%             |
| 64    | 55.38          | 120.90         | +118.3%            |
| 72    | 46.88          | 128.83         | +174.8%            |



- At width 8, the RCA is actually slightly faster than the CLA (308.55 vs 282.65 MHz). At this narrow width the extra carry-lookahead logic adds more delay than it saves.
- Starting from width 32, the CLA's parallel carry computation pays off, and the advantage grows significantly with width — reaching ~175% faster at width 72.
- The RCA degrades steeply with width because the carry must ripple through all bits. The CLA degrades much more gracefully, confirming its logarithmic critical path behavior.

## Max logic depth

| Width | RCA Max LUT Depth | CLA Max LUT Depth | CLA reduction vs RCA |
|-------|-------------------|-------------------|----------------------|
| 8     | 6                 | 6                 | 0%                   |
| 32    | 24                | 10                | -58.3%               |
| 64    | 45                | 14                | -68.9%               |
| 72    | 53                | 14                | -73.6%               |



- At width 8, both architectures share the same maximum logic depth (6 LUTs), consistent with the Fmax result where the RCA was marginally faster — there is no depth advantage to exploit at this width.
- From width 32 onward, the CLA's depth advantage becomes substantial and grows with width. By width 64/72 the CLA holds the depth constant at 14 LUTs while the RCA continues to grow linearly.
- The RCA depth scales linearly with width, as expected from its ripple-carry chain. The CLA depth grows much more slowly, reflecting its hierarchical carry structure — this directly explains the Fmax trends seen in the previous table.

## Critical paths

### RCA – Critical paths

| From Node | To Node     | w=8 | w=32 | w=64 | w=72 |
|-----------|-------------|-----|------|------|------|
| cin_q     | zero_f_q    | 6   | 24   | 45   | 53   |
| cin_q     | ov_f_q      | 6   | 24   | 45   | 53   |
| cin_q     | cout_q      | 6   | 24   | 45   | 53   |
| cin_q     | result_q[*] | 6   | —    | 45   | 53   |
| srcb_q[*] | zero_f_q    | 6   | 24   | 45   | 53   |
| srcb_q[*] | ov_f_q      | 6   | 24   | 45   | 53   |
| srcb_q[*] | cout_q      | 6   | 24   | 45   | 53   |
| srcb_q[*] | result_q[*] | 6   | —    | 45   | 53   |
| srca_q[*] | zero_f_q    | 6   | 24   | 45   | 53   |
| srca_q[*] | ov_f_q      | 6   | 24   | 45   | 53   |
| srca_q[*] | cout_q      | 6   | 24   | 45   | 53   |
| srca_q[*] | result_q[*] | 6   | —    | 45   | 53   |

- In the RCA, all critical paths originate from low-index inputs (cin\_q, srca\_q[0], srcb\_q[0], etc.) — bits that must propagate the carry through the entire chain to reach cout\_q, ov\_f\_q, zero\_f\_q, or the MSB of result\_q.

## CLA – Critical paths

| From Node | To Node  | w=8 | w=32 | w=64 | w=72 |
|-----------|----------|-----|------|------|------|
| cin_q     | zero_f_q | 6   | —    | —    | —    |
| srcb_q[*] | zero_f_q | 6   | 10   | 14   | 14   |
| srcb_q[*] | ov_f_q   | 6   | —    | —    | —    |
| srca_q[*] | zero_f_q | 6   | 10   | 14   | —    |
| srca_q[*] | ov_f_q   | 6   | —    | —    | —    |

- CLA collapses almost entirely to zero\_f\_q from width 32 onward, with depth plateauing at 14 between widths 64 and 72 — the carry chain is no longer the critical path; the OR reduction for the zero flag is.

## Other timing parameters

Both architectures share similar parameter values:

- **Latency:** 1 cycle (single cycle)
- **Throughput:** 1 operation per cycle
- **Slack:** Significantly negative for both architectures. Frequency was set to 1 GHz for analysis purposes and getting the maximum possible frequency, which was 308.55 MHz for the RCA when width=8.

## Conclusion

Ripple-carry adders are preferred narrow adders ( $\leq 8$  bits) or area-constrained designs where timing is not critical. At width 8, the RCA achieves higher Fmax than the CLA (308 vs 283 MHz) while using 37% fewer LUTs, making it the strictly better choice.

On the other hand, the best use case for carry look-ahead adders are wide adders ( $\geq 32$  bits) in performance-critical designs.

| Width         | Recommended | Rationale                                                                                                |
|---------------|-------------|----------------------------------------------------------------------------------------------------------|
| $\leq 8$ bits | RCA         | Higher Fmax, 37% fewer LUTs — strictly better at this width                                              |
| 32 bits       | CLA         | +57% Fmax at only +71% LUT cost; carry depth already 2.4× shallower                                      |
| 64 bits       | CLA         | +118% Fmax; depth advantage grows to 3.2×                                                                |
| 72 bits       | CLA         | +175% Fmax; depth has stabilized — the CLA scales to this width nearly for free when implemented in FPGA |